

PROGRAMMING IN HASKELL



Topic 12.1 – Monads

Recap on functor

1

```
class Functor f where  
  fmap :: (a -> b) -> f a -> f b
```

- `fmap (+2) [1,2,3]`
`[3,4,5]`
- `fmap (+2) (Just 1)`
`Just 2`
- `fmap (+2) Nothing`
`Nothing`

Recap on Applicative

2

- Generalise the idea of Functors to functions with more than one argument or parameter
- This gives rise to an Applicative Functor

```
class Functor f => Applicative where  
  pure :: a -> f a  
  ( <*> ) :: f (a-> b) -> fa -> fb
```

Data structure
of functions

Data
structure of
arguments

Data
structure of
results

Examples

3

```
➤ pure (+) <*> Just 1 <*> Just 2  
Just 3
```

```
➤ pure (+) <*> Nothing <*> Just 2  
Nothing
```

Applicative handles Errors (Nothing) correctly.

Introduce Monads - Example a simple evaluator

4

```
data Expr = Val Int | Div Expr  
e.g.  
e :: Expr  
e = Div (Val 6) (Val 3)
```



```
eval :: Expr -> Maybe Int
```

```
eval (Val n)      = n
```

```
eval (Div x y ) = eval x `div` eval y
```

Problem - how do we deal with division by zero (this would crash)

Need to refine program to deal with this

- How to capture failure

```
safeDiv :: Int -> Int -> Maybe Int
```

```
safeDiv _ 0    = Nothing
```

```
safeDiv n m    = Just (n `div` m)
```

Now, rewrite eval to use safeDiv

7

```
eval :: Expr -> Maybe Int
```

```
eval (Val n) = Just n
```

```
eval (Div x y) = case eval x of  
    Nothing -> Nothing  
    Just n -> case eval y of  
        Nothing -> Nothing  
        Just m -> safeDiv n m
```

Can we simplify?

Problem with Applicative

8

```
eval :: Expr -> Maybe Int
```

```
eval (Val n) = pure n
```

```
eval (Div x y) = pure safeDiv <*> eval x <*> eval y
```

Applicative needs `Int -> Int -> Int`

But we have `Int -> Int -> Maybe Int`

Maybe Int

Maybe Int

Monads - observe a common pattern and abstracting it out

9

```
eval :: Expr -> Maybe Int
```

```
eval (Val n) = Just n
```

```
eval (Div x y) = case eval x of
```

```
    Nothing -> Nothing
```

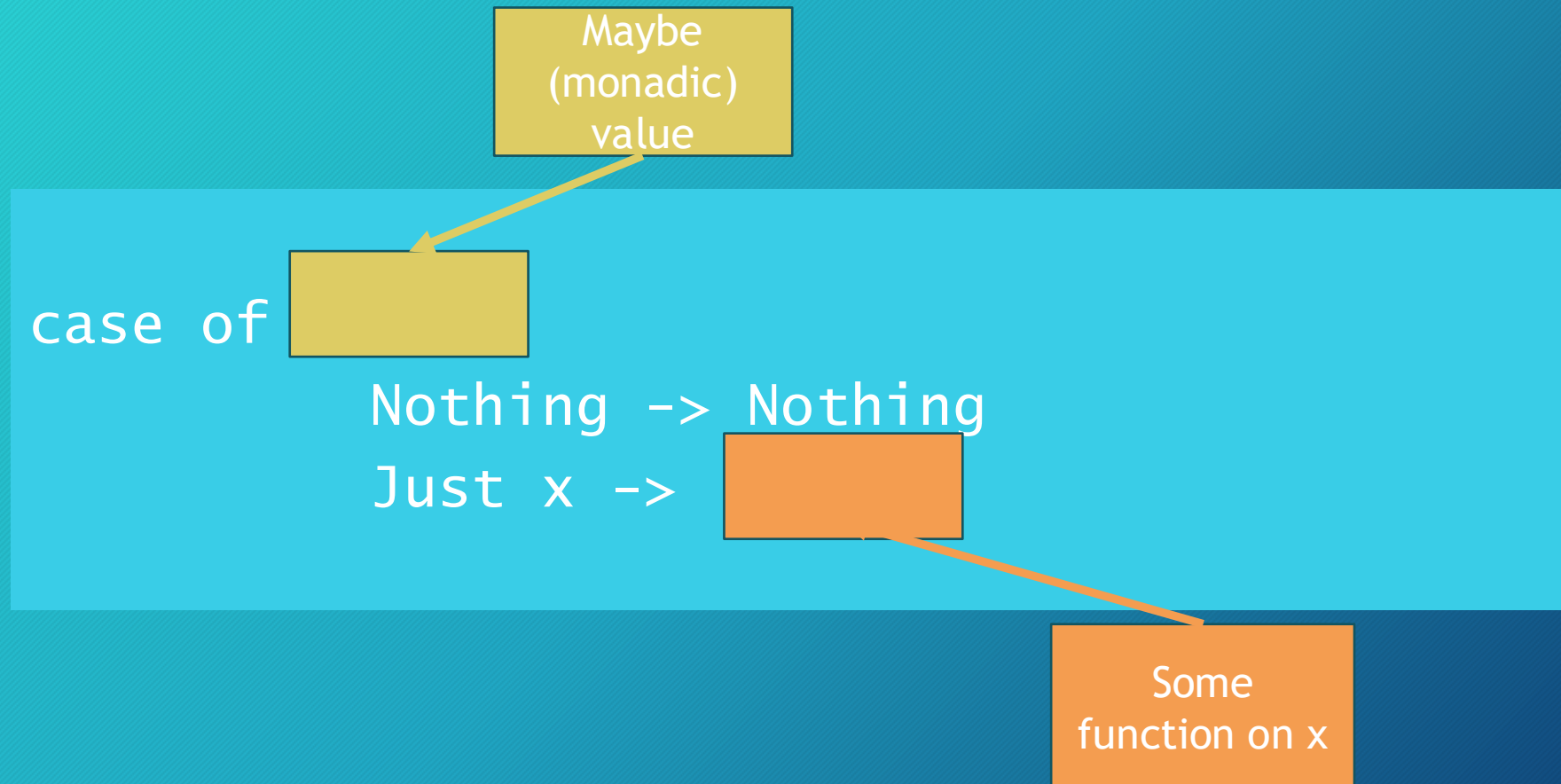
```
    Just n -> case eval y of
```

```
        Nothing -> Nothing
```

```
        Just m -> safeDiv n m
```


See pattern

10



See pattern

11

```
case mx of  
    Nothing -> Nothing  
    Just x -> f x
```

We want to turn
this into a
definition.

See pattern

12

```
case mx of  
  Nothing -> Nothing  
  Just x -> f x
```

We want to turn
this into a
definition.

```
mx >>= f = case mx of  
  Nothing -> Nothing  
  Just x -> f x
```

```
“into”      “bind”  
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
```


>>=

13



New evaluation function using >>=

14

```
eval :: Expr -> Maybe Int
eval (Val n)    = Just n
eval (Div x y) = eval x >>= (\n ->
                             eval y >>= (\m ->
                             safeDiv n m)    )
```


General pattern

15

```
m1 >>= \x1 ->  
m2 >>= \x2 ->  
m3 >>= \x3 ->  
:  
:  
mn >>= \xn ->  
f x1 x2 .. xn
```

Syntactic sugar

“This is what we use”

```
do x1 <- m1  
   x2 <- m2  
   x3 <- m3  
  
   xn <- mn  
f x1 x2 x3 .. xn
```


New evaluation function using do (final version)

16

```
eval :: Expr -> Maybe Int
eval (Val n)    = Just n
eval (Div x y) = do
    n <- eval x
    m <- eval y
    safeDiv n m
```


So now we are ready for Monads

17

Now that we have bind ($>>=$) we now can define this kind of applicative that has bind.

```
class Applicative m => Monad m where
  (>>= ) :: m a -> (a -> mb) -> mb
  return :: a -> ma
  return = pure
```


Example - Maybe - already have Functor, Applicative defs

18

```
data myMaybe a = Just a | Nothing deriving (Eq, Show)

instance Functor Maybe where
  fmap f (Just x) = Just (f x)
  fmap _ _       = Nothing
instance Applicative Maybe where
  pure = Just
  Nothing <*> _ = Nothing
  (Just f ) <*> something. = fmap f something
```


Example - Maybe - already have Functor, Applicative defs

19

```
data myMaybe a = Just a | Nothing deriving (Eq, Show)
```

```
instance Functor Maybe where
```

```
    fmap f (Just x) = Just (f x)
```

```
    fmap _ _       = Nothing
```

```
instance Applicative Maybe where
```

```
    pure = Just
```

```
    Nothing <*> _ = Nothing
```

```
    (Just f ) <*> something. = fmap f something
```


Example - Maybe - already have Functor, Applicative defs

```
data Maybe a = Just a | Nothing deriving (Eq, Show)

instance Functor Maybe where
  fmap f (Just x) = Just (f x)
  fmap _ _       = Nothing
instance Applicative Maybe where
  pure = Just
  Nothing <*> _ = Nothing
  (Just f ) <*> something. = fmap f something
```

```
instance Monad Maybe where
```

```
-- (>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b)
```

```
Nothing >>= f  = Nothing
```

```
Just f  >>= f = f x
```


Example - Lists

21

```
instance Monad [] where
  -- (>>=) :: [a] -> (a -> [b] ) -> [b]
  Nothing >>= f  = Nothing
  xs >>= f  = concat  map f xs
```

Note the
concat

or

```
instance Monad [] where
  -- (>>=) :: [a] -> (a -> [b] ) -> [b]
  Nothing >>= f  = Nothing
  xs >>= f  = [y | x <- xs, y <- f x]
```


Example - using List monads

22

```
➤ pairs [1,2] [3,4]    -- cartesian product  
[(1,3), (1,4), (2,3), (2,4)]
```

or

```
pairs :: [a] -> [b] -> [(a,b)]  
pairs xs ys = do x <- xs  
                  y <- ys  
                  return (x,y)
```


Monads

23

A monad is defined by 3 things:

- A type constructor m
- A function `return`
- An operator `(>>=)` – read as “bind”

Originally, these were introduced for IO, but they can be much more powerful.

```
return :: a -> m a
(>>=)  :: m a -> (a -> m b) -> m b
```

The function and operator are methods of the Monad typeclass and have types themselves, which are required to obey 3 laws (more later):

Monad laws

24

Monads are required to obey 3 laws:

```
m  >>= return      = m
return x >>= f       = f x
(m >>= f) >>= g      = m >>= (\x -> x >>= g) -- associativity
```

Monads originally come from a branch of mathematics called Category Theory.

(Fortunately, it is entirely unnecessary to understand category theory in order to understand and use monads in Haskell.)

Monadic Programming

25

Having designed a Monad (i.e. 'I need a Monad with this shape'), we define an instance of a Monad.

This involves essentially writing what

1. `return` and
2. `>>=` mean in this Monad

(this assumes you have written instances for Functor and Applicative)

Examples of monadic programming - 1 - fmap

26

```
--- fmap using IO  
fmap :: (a -> b) -> IO a -> IO b
```

Given

- a function $f :: a \rightarrow b$ and
- an IO interaction $m :: IO\ a$,

run the interaction,

- then apply f to the result, and
- wrap it back in IO.

fmap

27

```
fmap :: (a -> b) -> IO a -> IO b
```

Given

- a function $f :: a \rightarrow b$ and
- an IO interaction $m :: IO\ a$,

We want to:

- run the interaction,
- then apply f to the result, and
- wrap it back in IO.

```
fmap f m =  
    do  
        x <- m  
        return(f x)
```


Examples of monadic programming- 2 - using recursion - repeat

28

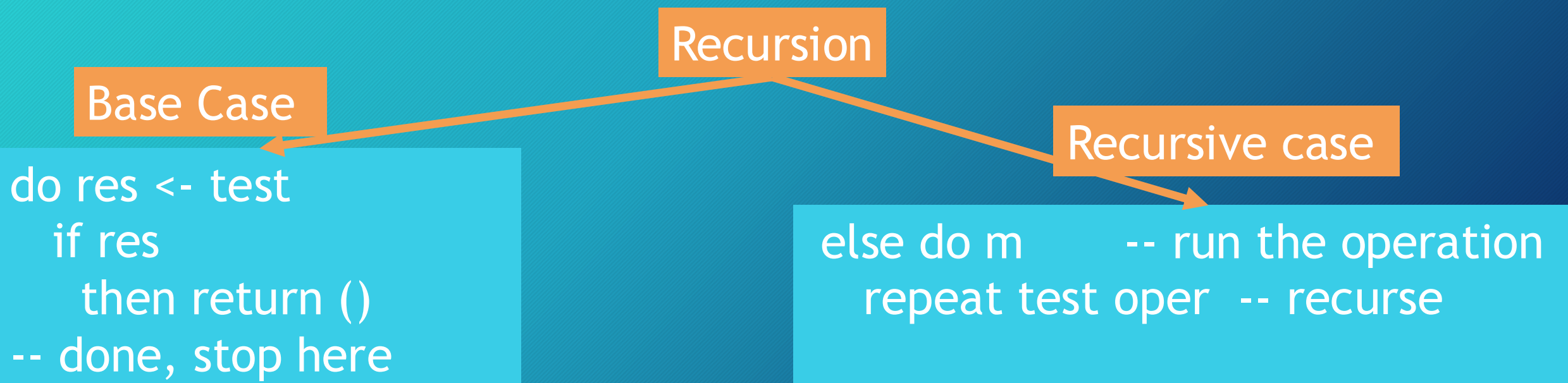
```
repeat :: IO Bool -> IO () -> IO ()
```

So that

repeat test oper

keeps running oper until test returns True.

Recursion



```
graph TD; Recursion[Recursion] --> BaseCase[Base Case]; Recursion --> RecursiveCase[Recursive case];
```

Base Case

```
do res <- test
  if res
    then return ()
  -- done, stop here
```

Recursive case

```
else do m      -- run the operation
      repeat test oper -- recurse
```


Examples of monadic programming- 2 - using recursion - repeat

29

Full solution:

```
repeat :: IO Bool -> IO () -> IO ()
```

```
repeat test oper
```

```
  = do res <- test
```

```
    if res
```

```
      then return ()
```

```
      else do oper
```

```
        repeat test m
```

run test, bind result to res

if True → done (base case)

if False → run oper then recurse

